

**MINISTERUL EDUCAȚIEI ȘI CERCETĂRII
al REPUBLICII MOLDOVA**

UNIVERSITATEA TEHNICĂ A MOLDOVEI

**FACULTATEA CALCULATOARE, INFORMATICĂ
ȘI MICROELECTRONICĂ**

DEPARTAMENTUL INFORMATICĂ ȘI INGINERIA SISTEMELOR

RAPORT

**Aplicație client - Server de gestiunarea a procesului de învățământ în
școală**

st. gr. CR - 231

conf. univ., dr.

A efectuat:

Pricop Alexandru

A verificat:

Rotaru Lilia

Chișinău - 2026

CUPRINS

INTRODUCERE	1
1. ANALIZA PIEȚII PENTRU MANAGEMENTUL ACTIVITĂȚILOR ȘCOLARE.....	4
1.1. Analiza aplicațiilor de pe piața	4
1.2 Compararea aplicațiilor	7
2. ANALIZA TEHNOLOGIILOR UTILIZATE ÎN APLICAȚIE (CLIENT – SERVER – ADMIN).....	10
2.1 Client mobil — Dart + Flutter	10
2.2 Management de stare și arhitectură pe funcționalități — GetX + GetIt	10
2.3 Comunicație cu serverul și tratarea erorilor — Dio + interceptori	10
2.4 Securitatea sesiunii — Secure Storage (token)	10
2.5 Funcționare offline și sincronizare — OfflineActionHandler	11
2.6 Server (Backend API) — Python + FastAPI	11
2.7 Persistență și modelarea datelor	11
2.8 Componenta Admin (management)	11
3. CREAREA ALGORITMULUI ȘI A PROGRAMULUI	12
3.1 Descrierea schemei de funcționare a programului	12
3.2 Crearea algoritmul programului	13
3.3 Descrierea programului	14
2. Partea Client (Aplicația Mobilă)	15
4. Baza de Date	16
CONCLUZIA	17
BIBLIOGRAFIA	18

INTRODUCERE

În prezent, instituțiile educaționale, în special universitățile, se confruntă cu dificultăți semnificative în gestionarea și comunicarea eficientă a orarului către studenți și profesori. Soluțiile tradiționale, precum afișarea orarelor în format fizic sau publicarea lor pe platforme web statice, au limite semnificative în termeni de accesibilitate, actualizare rapidă și personalizare a informațiilor. Aceste metode nu reușesc să răspundă nevoilor dinamice ale utilizatorilor, care necesită acces instantaneu și flexibil la informații actualizate.

În era digitală, eficiența și accesibilitatea informațiilor sunt esențiale pentru buna funcționare a oricărei instituții educaționale. Studenții și profesorii au nevoie de soluții moderne care să le permită să acceseze rapid și ușor informațiile necesare pentru planificarea și organizarea activităților academice. O soluție digitală eficientă trebuie să ofere nu doar vizualizarea orarelor în timp real, ci și funcționalități adiționale de căutare, filtrare și export, cu o interfață prietenoasă pentru utilizatori.

Dezvoltarea unei aplicații client-server de vizualizare a orarului vine ca răspuns la aceste nevoi. O astfel de aplicație poate îmbunătăți semnificativ comunicarea și organizarea în cadrul instituțiilor educaționale, reducând timpul și efortul necesar pentru gestionarea orarelor și crescând satisfacția utilizatorilor. Prin implementarea unei soluții moderne, bazate pe o arhitectură client-server robustă, se poate asigura accesul rapid și securizat la informațiile despre cursuri, profesori și săli.

În plus, o aplicație bine concepută va include notificări automate pentru modificările de ultim moment ale orarului, asigurându-se astfel că utilizatorii sunt mereu la curent cu cele mai recente informații. De asemenea, se va pune un accent deosebit pe securitatea datelor și pe protecția informațiilor personale ale utilizatorilor, respectând toate standardele și reglementările în vigoare.

Astfel, dezvoltarea și implementarea unei aplicații client-server de vizualizare a orarului reprezintă un pas important către modernizarea și eficientizarea proceselor educaționale, oferind o soluție adaptată nevoilor actuale ale studenților și profesorilor.

Deci, în era tehnologiei, instituțiile educaționale pot beneficia considerabil de pe urma implementării unor sisteme digitale care să ofere acces în timp real la informațiile orarului, să permită actualizări rapide și precise, și să sprijine personalizarea programului în funcție de nevoile individuale ale utilizatorilor.

1. ANALIZA PIETII PENTRU MANAGEMENTUL ACTIVITAȚILOR ȘCOLARE.

1.1. Analiza aplicațiilor de pe piata

În ultimii ani, digitalizarea procesului educațional a devenit o prioritate pentru instituțiile de învățământ din întreaga lume. Apariția aplicațiilor și platformelor educaționale a contribuit semnificativ la îmbunătățirea comunicării dintre profesori, elevi și părinți, precum și la eficientizarea procesului de învățare.

Aplicație educațională Google Classroom

Google Classroom, a cărei siglă este prezentată în Figura 1.1, este o platformă educațională dezvoltată de Google, destinată să faciliteze gestionarea cursurilor, comunicarea între profesori și elevi și organizarea activităților școlare online. Aceasta oferă un mediu centralizat pentru distribuirea temelor, evaluarea elevilor și colaborarea digitală între participanții la procesul educațional. Mai jos sunt prezentate câteva din caracteristicile de bază ale aplicației:

Interfața utilizatorului (UI)

Google Classroom are o interfață intuitivă și simplă, accesibilă atât pe desktop, cât și pe mobil. Profesorii pot crea clase virtuale, pot distribui teme, pot posta anunțuri și pot vizualiza progresul elevilor. Elevii pot accesa materialele școlare, pot preda temele și pot primi feedback direct de la profesori. UI-ul facilitează navigarea rapidă între cursuri și clase.

Partea client:

Partea client este disponibilă prin aplicații web și mobile. Utilizatorii pot accesa cursurile, teme și resurse direct din aplicație. Elevii pot încărca teme și fișiere, iar profesorii pot evalua și comenta lucrările online. Aplicația permite notificări în timp real pentru teme noi, anunțuri sau mesaje din clasă.

Partea server:

Google Classroom funcționează pe serverele distribuite ale Google, asigurând performanță și disponibilitate constantă. Serverele gestionează autentificarea utilizatorilor, stocarea datelor și sincronizarea informațiilor între toate dispozitivele conectate. Funcționalități precum backup-ul automată și controlul versiunilor materialelor școlare sunt integrate la nivel server.

Avantaje:

- Integrare completă cu alte servicii Google (Drive, Docs, Calendar);
- Permite gestionarea temelor, evaluărilor și comunicarea într-un singur loc;
- Notificări și actualizări în timp real; Gratuit pentru școli și instituții educaționale;
- Posibilitate de import/export a materialelor și rapoartelor.

Dezavantaje:

- Necesită cont Google pentru autentificare;
- Personalizarea platformei este limitată;
- Depinde de conexiunea la internet;
- Nu are funcționalități complexe pentru managementul complet al școlilor (ex. gestionare director, clase multiple sau statistici detaliate fără integrare suplimentară);
- Mai puțin potrivit pentru școlile care nu folosesc ecosistemul Google.

Aplicație educațională *ClassDojo*

ClassDojo, a cărei siglă este prezentată în Figura 1.3, este o aplicație mobilă și web destinată să faciliteze colaborarea dintre profesori, elevi, axându-se pe comunicare, comportament pozitiv și implicarea familiei în procesul educațional. Platforma transformă modul în care școlile gestionează activitățile zilnice și relația cu părinții.

Interfața utilizatorului (UI):

ClassDojo are o interfață prietenoasă și colorată, ușor de utilizat pe desktop și mobil. Profesorii pot crea clase, pot urmări comportamentul elevilor și pot acorda puncte de recompensă. Elevii pot vedea progresul, pot primi feedback și pot comunica direct cu profesorii. UI-ul permite acces rapid la rapoarte și anunțuri.

Partea client:

Aplicația este disponibilă pe web și mobil. Profesorii pot posta activități, teme și anunțuri, iar părinții primesc notificări în timp real despre progresul copilului. Elevii pot primi mesaje motivante și pot interacționa cu profesorii și colegii într-un mod sigur și controlat.

Partea server:

Serverele ClassDojo gestionează conturile utilizatorilor, stochează datele despre clase, elevi și comportament și sincronizează informațiile între toate dispozitivele conectate. Sistemul este scalabil și permite acces rapid la informații fără întreruperi.

Avantaje:

- Interfață intuitivă, ușor de folosit pentru profesori și elevi;
- Integrare completă cu alte servicii Google (Drive, Docs, Calendar);
- Permite gestionarea temelor, evaluărilor și comunicarea într-un singur loc;
- Notificări și actualizări în timp real;
- Gratuit pentru școli și instituții educaționale;
- Posibilitate de import/export a materialelor și rapoartelor.

Dezavantaje:

- Necesită cont Google pentru autentificare;
- Personalizarea platformei este limitată;
- Depinde de conexiunea la internet;
- Nu are funcționalități complexe pentru managementul complet al școlilor (ex. Gestionare director, clase multiple sau statistici detaliate fără integrare suplimentară);
- Mai puțin potrivit pentru școlile care nu folosesc ecosistemul Google.

Platformă Educațională Adservio

Adservio este o platformă educațională românească folosită în unele școli din Republica Moldova și România, destinată gestionării complete a procesului educațional. Platforma permite administrarea catalogelor, evidența notelor și absențelor, gestionarea orarului și comunicarea între profesori, elevi.

Interfața utilizatorului (UI):

Adservio oferă o interfață clară și profesională, orientată către utilizatori de toate vârstele. Profesorii pot gestiona catalogul clasei, pot adăuga note și absențe, iar elevii pot accesa informațiile în timp real. Interfața include dashboard-uri cu statistici și rapoarte

Partea client:

Platforma este disponibilă atât pe web, cât și pe mobil (Android și iOS). Profesorii pot introduce note și teme, și elevii pot vizualiza progresul academic. Toate datele sunt sincronizate în timp real, iar notificările sunt trimise automat pentru evenimente relevante.

Partea server:

Serverele Adservio gestionează toate datele despre școală, clase, profesori, elevi. Sistemul permite import/export de date, backup automat și control al accesului pe roluri. Arhitectura este scalabilă pentru mai multe școli și clase simultan.

1.2 Compararea aplicațiilor

Aplicațiile Google Classroom, ClassDojo și Adservio reprezintă soluții digitale diferite ca funcționalitate, scop și abordare tehnologică. Deși toate contribuie la digitalizarea procesului educațional, fiecare se adresează unui segment specific de nevoi din cadrul școlilor.

Domeniul de utilizare:

Google Classroom este orientat către gestionarea cursurilor, temelor și colaborării academice între profesori și elevi.

ClassDojo se concentrează în principal pe comunicarea dintre profesori, elevi și părinți, având un accent puternic pe motivarea elevilor și managementul comportamental.

Adservio oferă o soluție completă de administrare a întregii școli, incluzând cataloage, orare, evidență note/absențe și comunicare administrativă.

Comparativ:

Classroom → predare și teme

ClassDojo → comportament și comunicare

Adservio → management complet școlar

Interfața utilizatorului (UI)

Google Classroom are un design minimalist, axat pe simplitate și organizare pe cursuri.

ClassDojo utilizează o interfață colorată, prietenoasă, destinată în special claselor primare, favorizând interacțiunea și motivația.

Adservio are o interfață profesională, orientată către administrarea datelor academice.

Comparativ:

Classroom → simplă, structurată pentru lecții

ClassDojo → jucăușă, atractivă pentru copii

Adservio → formală, orientată spre rapoarte și statistici

Partea client:

Google Classroom permite gestionarea temelor, vizualizarea cursurilor, colaborarea în Google Docs și primirea notificărilor.

ClassDojo oferă funcții de comunicare, acordare de puncte, postare de anunțuri și vizualizare a progresului comportamental.

Adservio permite accesul la catalog, note, absențe, teme, orar și comunicarea cu școala.

Comparativ:

Classroom → teme, materiale, feedback

ClassDojo → puncte, comportament, mesaje către părinți

Adservio → note, absențe, orar, teme, rapoarte

Partea server:

Google Classroom funcționează pe infrastructura Google Cloud, asigurând sincronizare rapidă și disponibilitate globală.

ClassDojo utilizează servere scalabile pentru gestionarea comportamentului elevilor, conturi și comunicare în timp real.

Adservio are o arhitectură care stochează date academice la nivel local (România), cu acces controlat pentru profesori, elevi și părinți.

Comparativ:

Classroom → servere Google, performanță globală

ClassDojo → servere cloud scalabile pentru interacțiuni rapide

Adservio → servere dedicate pentru școli, accent pe date academice complexe

Caracteristică	Google Classroom	ClassDojo	Adservio
Gestionare teme	✓	✗	✓
Evaluare academică	✓	✗	✓
Management comportament	✗	✓	✗
Comunicare cu părinții	✓	✓	✓
Catalog digital	✗	✗	✓
Statistici și rapoarte	Limitate	Simplificat	Detaliate
Orare, clase, administrare școală	✗	✗	✓

Avantaje:

- Google Classroom: gratuit, integrat în ecosistemul Google, simplu și rapid.
- ClassDojo: atractiv pentru elevi, excelent pentru comunicare și motivație.
- Adservio: soluție completă pentru management educațional, catalog digital și rapoarte complexe.

Dezavantaje:

- ✓ Google Classroom: necesită cont Google, personalizare redusă, nu conține catalog școlar complet.
- ✓ ClassDojo: nu gestionează note, absențe sau documente administrative; orientat mai mult spre comportament.
- ✓ Adservio: poate fi costisitor pentru școli, interfața mai complexă, depinde de adoptarea școlii la nivel administrativ.

Concluzie comparativă:

Cele trei aplicații analizate acoperă nevoi diferite din domeniul educațional:

- ❖ Google Classroom este ideal pentru predare online și organizarea materialelor școlare.
- ❖ ClassDojo este potrivit pentru școlile care urmăresc implicarea elevilor și comunicarea cu părinții.
- ❖ Adservio este cel mai complet sistem, oferind funcții administrative esențiale pentru o gestionare academică profesională.
- ❖ Combinate, aceste aplicații ar putea acoperi întreg procesul educațional, însă în practică școlile aleg platforma care se potrivește cel mai bine nevoilor lor specifice.

2. ANALIZA TEHNOLOGIILOR UTILIZATE ÎN APLICAȚIE (CLIENT – SERVER – ADMIN)

Dezvoltarea aplicației urmează o arhitectură de tip client–server–admin, în care:

- clientul mobil gestionează interfața și fluxurile utilizatorului,
- serverul API centralizează regulile de business și accesul la date,
- componenta admin oferă funcții de administrare (utilizatori, clase, resurse etc.).
- Stack-ul tehnologic este mixt, conform proiectului: Flutter/Dart pentru client, Python/FastAPI pentru server [1][8].

2.1 Client mobil — Dart + Flutter

Clientul este implementat în Dart, folosind Flutter pentru dezvoltarea interfeței grafice cross-platform (Android/iOS) [6]. Flutter permite realizarea unei interfețe moderne și reactive, cu performanță bună și cu un singur cod sursă pentru mai multe platforme[5] [7].

2.2 Management de stare și arhitectură pe funcționalități — GetX + GetIt

În client este utilizat GetX pentru:

- Stări reactive (ex. Rx, Rxn, liste observabile),
- Controllers care coordonează încărcarea datelor și actualizarea UI-ului,
- Navigație și mesaje către utilizator (ex. snackbars).
- Injectarea dependențelor este realizată cu GetIt, prin care controller-ele primesc use-case-uri și data source-uri fără a crea manual instanțe în UI.

2.3 Comunicație cu serverul și tratarea erorilor — Dio + interceptori

Comunicarea client–server se realizează prin Dio, un client HTTP avansat care suportă:

- Interceptori pentru request/response,
- Tratarea structurată a erorilor (ex. DioException),
- Upload de fișiere (ex. avatar),
- Integrare ușoară cu autentificare bazată pe token.
- La erori de autentificare (ex. cod 401), fluxul aplicației poate redirecționa automat utilizatorul către ecranul de login.

2.4 Securitatea sesiunii — Secure Storage (token)

Token-ul de autentificare este stocat folosind un serviciu de secure storage, pentru a evita păstrarea informațiilor sensibile în clar. La expirarea sau invalidarea sesiunii, token-ul este șters, iar utilizatorul este redirecționat către autentificare.

2.5 Funcționare offline și sincronizare — OfflineActionHandler

Un element important al clientului este suportul offline-first:

- operațiile de tip create (ex. prezențe, teme, note) pot fi executate imediat dacă există conexiune;
- dacă aplicația este offline, acțiunea este salvată local ca operație în coadă și va fi sincronizată ulterior prin mecanisme de tip SyncOperation.
- Acest mecanism crește robustețea aplicației în medii cu conexiune instabilă la internet.

2.6 Server (Backend API) — Python + FastAPI

Serverul este implementat în Python, folosind framework-ul FastAPI, cu pachete gestionate prin pip. FastAPI este potrivit pentru dezvoltarea de API-uri moderne datorită [1]:

- performanței ridicate (bazat pe ASGI),
- validării automate a request-urilor prin modele Pydantic,
- generării automate de documentație OpenAPI/Swagger,
- organizării clare pe routere, endpoint-uri și straturi (schemas, services).

Serverul expune endpoint-uri consumate de client pentru domenii precum: autentificare, profesori, elevi, clase, note, teme, prezențe, orar și materiale, inclusiv operații de încărcare fișiere unde este necesar.

2.7 Persistență și modelarea datelor

Persistența este gestionată pe server, în funcție de configurația proiectului. Pentru un sistem școlar sunt esențiale:

- consistența tranzacțiilor pentru date critice (note, prezențe),
- integritatea relațiilor dintre entități (clase–elevi–profesori),
- interogări eficiente pentru dashboard, orar și rapoarte.

2.8 Componenta Admin (management)

Zona de admin are rolul de a gestiona entitățile și operațiile administrative (utilizatori, clase, orare, resurse). De regulă, componenta admin consumă aceleași endpoint-uri FastAPI (sau endpoint-uri dedicate), astfel încât regulile de business și validările să rămână centralizate pe server.

3. CREAREA ALGORITMULUI ȘI A PROGRAMULUI

3.1 Descrierea schemei de funcționare a programului

Schema din figura 3.1 ilustrează arhitectura aplicației, evidențiind principalele componente și modul de comunicare dintre acestea. Sistemul include cinci categorii de utilizatori: Director, Profesor, Elev și Admin, fiecare având un rol bine definit în cadrul aplicației.

Profesorul și elevul utilizează aplicația prin intermediul clientului, de unde pot accesa informațiile puse la dispoziție de sistem, cum ar fi orarul, notele, situația absențelor, anunțurile sau alte date relevante pentru rolul lor. Toate cererile efectuate de acești utilizatori sunt transmise către server, care procesează solicitările și comunică direct cu baza de date pentru a prelua sau actualiza informațiile necesare.

În schimb, Adminul dispune de acces direct la baza de date. Acesta poate efectua operațiuni privilegiate precum adăugarea, modificarea sau ștergerea datelor interne, fără a trece prin server. Această abordare este utilizată pentru a reduce sarcina serverului, permițând ca activitățile administrative complexe să fie realizate direct asupra bazei de date, fără intermediere suplimentară.

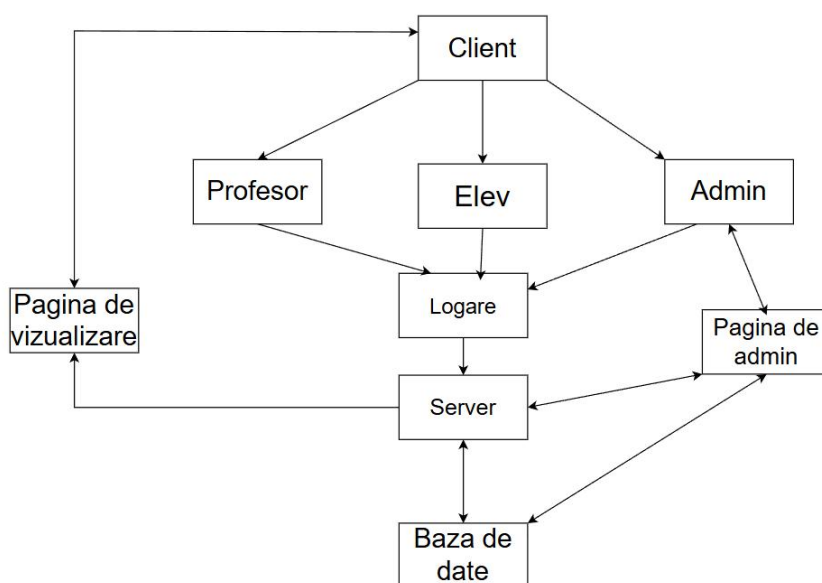


Figura 3.1. Schema de funcționare a programului

Serverul are rolul central de procesare: acesta primește cererile de la client, interacționează cu baza de date (Postgresql) și returnează răspunsurile corespunzătoare către utilizatori. Baza de date reprezintă depozitul principal de informații, asigurând coerența și actualizarea în timp real a datelor accesate de utilizatori.

Fluxul de date se desfășoară astfel: Profesorul și Elevul transmit cereri către client, clientul le direcționează către server, iar serverul comunică cu baza de date pentru a executa operațiunile solicitate. În paralel, Adminul interacționează direct cu baza de date pentru gestionarea structurii interne a informațiilor. Această arhitectură evidențiază modul în care sistemul facilitează un schimb eficient de date, oferind utilizatorilor acces rapid la informații actualizate și funcționând într-un mod optimizat pentru performanță.

3.2 Crearea algoritmul programului

Schema din figura 3.2 prezintă fluxul de autentificare al aplicației, evidențiind pașii parcurși de utilizator și modul în care serverul gestionează verificările necesare înainte de a permite accesul în sistem. Procesul începe din momentul în care **clientul inițiază acțiunea de logare**, introducând datele de autentificare.

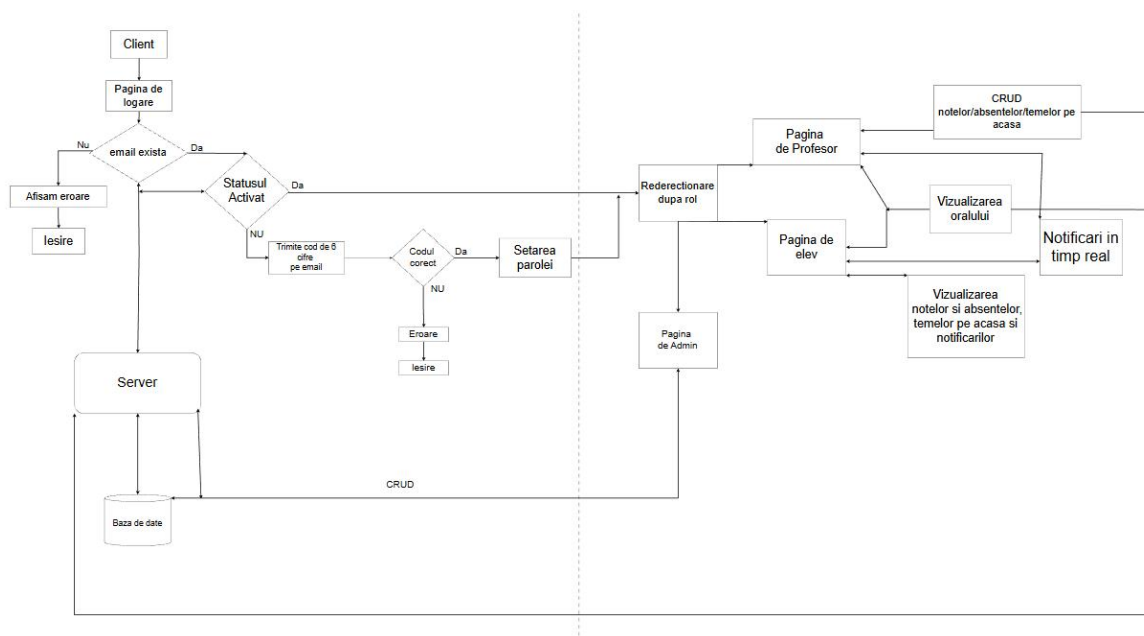


Figura 3.2. Algoritmul programului

Primul pas implică verificarea existenței emailului în baza de date. Dacă adresa introdusă nu este găsită, aplicația afișează utilizatorului un mesaj prin care îl informează că acest cont nu există în sistem. În cazul în care emailul este valid, fluxul continuă cu verificarea statusului contului, pentru a determina dacă utilizatorul și-a activat deja profilul și dacă are o parolă setată. Dacă statusul utilizatorului indică faptul că acesta nu are încă o parolă configurată, aplicația îl direcționează către procesul de setare a parolei. În situația în care contul este activ și valid, serverul obține și verifică rolul asociat utilizatorului. Pe baza rolului, serverul stabilește care este destinația finală a utilizatorului în aplicație.

Dacă rolul utilizatorului este admin, fluxul este redirecționat către panoul de administrare, unde acesta are acces la operațiuni CRUD și gestionarea datelor sistemului. În schimb, dacă utilizatorul nu este admin, acesta este direcționat către pagina principală a aplicației, unde poate accesa funcționalitățile specifice rolului său (elev, profesor).

Pe tot parcursul procesului, serverul joacă rolul central, deoarece primește, validează și procesează datele, comunicând permanent cu baza de date pentru verificarea informațiilor și returnarea rezultatelor către client. Această arhitectură asigură un flux coerent, sigur și controlat al autentificării, evitând accesul neautorizat și garantând că fiecare utilizator este direcționat corect în funcție de statusul și rolul său.

3.3 Descrierea programului

Aplicația dezvoltată, este un sistem client–server destinat gestionării și consultării informațiilor din cadrul unei instituții de învățământ preuniversitar. Sistemul este împărțit în două componente principale: un server API și un client mobil, fiecare având responsabilități clare în arhitectura aplicației. Serverul reprezintă nucleul aplicației și implementează logica de business, validările și regulile de acces. Acesta este realizat în Python folosind FastAPI, iar persistența datelor este gestionată prin SQLAlchemy (async), cu o bază de date PostgreSQL. Serverul expune endpoint-uri REST care permit autentificarea utilizatorilor și accesul la resurse precum utilizatori, profesori, elevi, clase, orar/schedule și alte entități specifice școlii. În plus, serverul aplică restricții pe roluri (ex. elev/profesor/director/administrator) pentru a permite doar operațiunile corespunzătoare fiecărui tip de utilizator. Clientul este o aplicație dezvoltată în Dart/Flutter, destinată utilizatorilor finali [4][12].

Aceasta oferă o interfață grafică pentru vizualizarea datelor provenite de la server și pentru interacțiuni specifice rolului autentificat. Spre exemplu, profesorii pot consulta clasele asociate și informațiile necesare activității didactice, iar directorul are un dashboard dedicat pentru rapoarte și acces la liste extinse (de exemplu, toate clasele din școala sa). Comunicarea cu serverul este realizată prin cereri HTTP, folosind biblioteca Dio, iar aplicația afișează date precum liste, detalii și rapoarte într-un mod accesibil pe dispozitive mobile. Cele două module sunt conectate prin intermediul bazei de date PostgreSQL, care funcționează ca depozit central pentru informații. Structura bazei de date este relațională și modelează entitățile principale ale școlii (ex. utilizatori, profesori, elevi, clase, programări/orar), precum și relațiile dintre acestea.

Astfel, datele sunt coerente, ușor de interogată și pot fi sincronizate între utilizatori în funcție de drepturile de acces. Arhitectura client–server asigură o separare clară a responsabilităților: serverul controlează logica și securitatea, iar clientul se ocupă de prezentare și experiența utilizatorului. Această separare permite scalarea independentă a componentelor, mentenanță mai ușoară și extinderea ulterioară a aplicației (de exemplu, adăugarea de module suplimentare pentru rapoarte, catalog, notificări sau administrare avansată).

1. Partea de Administrare și Management (Administrator și Director)

În cadrul aplicației SchoolApp, rolurile de tip administrator și director gestionează datele principale ale sistemului. Aceste roluri au dreptul de a crea, modifica și șterge entități precum: utilizatori, profesori, elevi, clase și orar.

Interacțiunea cu sistemul se realizează prin aplicația client Flutter, care transmite cereri HTTP către backend. Serverul este implementat în Python folosind framework-ul FastAPI, iar validarea datelor se face prin modele Pydantic [12].

Înainte de aplicarea oricărei modificări, serverul verifică:

- corectitudinea datelor introduse;
- drepturile utilizatorului (autorizare pe bază de rol);
- consistența relațiilor dintre entități (de exemplu: asocierea profesor–clasă sau școală–utilizator).

Separarea „scope-ului” datelor permite:

- directorului să vizualizeze toate clasele din școala sa;
- profesorului să acceseze doar clasele care îi sunt asociate.

2. Partea Client (Aplicația Mobilă)

Partea de client a aplicației este implementată în Flutter (Dart) și este destinată utilizatorilor finali: elev, profesor și director.

Clientul:

- gestionează autentificarea utilizatorului;
- consumă endpoint-urile REST oferite de server;
- afișează datele prin ecrane de tip dashboard, liste și pagini de detalii.

Interfața se adaptează automat în funcție de rolul utilizatorului:

- elev: vizualizare clasă și orar;
- profesor: profil, clase asociate și orar;
- director: dashboard și acces extins la datele școlii.

3. Partea Serverului:

Serverul reprezintă componenta centrală a aplicației și este construit folosind FastAPI.

Principalele responsabilități sunt:

- definirea și rutarea endpoint-urilor REST;
- autentificarea și autorizarea utilizatorilor; validarea datelor cu Pydantic;
- accesul asincron la baza de date folosind SQLAlchemy async.

În timpul dezvoltării au fost întâlnite două probleme tipice:

- MissingGreenlet, cauzată de folosirea lazy loading în context asincron;
- Soluția a fost utilizarea mecanismelor de eager loading (selectinload, joinedload);
- ResponseValidationError, cauzată de neconcordanțe între schema Pydantic și structura răspunsului JSON [11].

4. Baza de Date

Aplicația folosește PostgreSQL ca sistem de gestiune a bazei de date. Structura este relațională și include următoarele entități [9]:

- ✓ users,
- ✓ teachers,
- ✓ students,
- ✓ classes,
- ✓ schedules.

Toate operațiile sunt realizate exclusiv prin server, folosind ORM-ul SQLAlchemy.

CONCLUZIA

În cadrul acestui proiect a fost proiectată și implementată aplicația SchoolApp, un sistem informatic modern pentru gestionarea activității școlare, bazat pe o arhitectură client–server și tehnologii actuale din domeniul dezvoltării software.

Aplicația oferă suport pentru mai multe tipuri de utilizatori (elev, profesor, director, administrator) și permite gestionarea centralizată a datelor despre utilizatori, clase, profesori și orar. Separarea clară între interfața client (Flutter), server (FastAPI) și baza de date (PostgreSQL) asigură o arhitectură scalabilă, sigură și ușor de întreținut.

Un aspect important al proiectului este implementarea controlului accesului pe bază de roluri, care garantează că fiecare utilizator poate accesa doar datele corespunzătoare poziției sale în sistem (de exemplu, profesorul vede doar clasele sale, iar directorul vede toate clasele din școală).

Utilizarea SQLAlchemy async și a modelelor Pydantic asigură atât performanță, cât și siguranță în manipularea datelor. Problemele întâlnite în timpul dezvoltării (precum MissingGreenlet și ResponseValidationError) au fost rezolvate prin aplicarea corectă a mecanismelor de eager loading și prin alinierea strictă a schemelor de date [4][11].

În concluzie, SchoolApp demonstrează cum poate fi realizată o aplicație educațională modernă, robustă și extensibilă, care poate fi dezvoltată în continuare prin adăugarea unor funcționalități precum: catalog electronic, mesagerie internă, rapoarte statistice și integrare cu sisteme externe.

BIBLIOGRAFIA

1. **FastAPI Documentation** - <https://fastapi.tiangolo.com>
2. **FastAPI Advanced User Guide** - <https://fastapi.tiangolo.com/advanced/>
3. **FastAPI Security** - <https://fastapi.tiangolo.com/tutorial/security/>
4. **SQLAlchemy Documentation** - <https://docs.sqlalchemy.org>
5. **Flutter Documentation** - <https://docs.flutter.dev>
6. **Flutter Widgets Catalog** - <https://docs.flutter.dev/ui/widgets>
7. **Flutter Navigation and Routing** - <https://docs.flutter.dev/ui/navigation>
8. **Dart Language Documentation** - <https://dart.dev/guides>
9. **PostgreSQL Documentation** - <https://www.postgresql.org/docs/>
10. **PostgreSQL Indexes** - <https://www.postgresql.org/docs/current/indexes.html>
11. **Pydantic Documentation** - <https://docs.pydantic.dev>
12. **Python Official Documentation** - <https://docs.python.org/3/>
13. **OpenAPI Specification** - <https://swagger.io/specification/>